

Vertex AI：分散式超參數調整

來源：<https://codelabs.developers.google.com/vertex-training-200?hl=zh-tw>

步驟數：8

目錄

1. [總覽](#)
2. [Vertex AI 簡介](#)
3. [用途總覽](#)
4. [設定環境](#)
5. [編寫訓練程式碼](#)
6. [將程式碼容器化](#)
7. [啟動超參數調整工作](#)
8. [清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將瞭解如何使用 Vertex AI 進行超參數調整和分散式訓練。雖然本實驗室使用 TensorFlow 撰寫模型程式碼，但這些概念也適用於其他機器學習架構。

課程內容

內容如下：

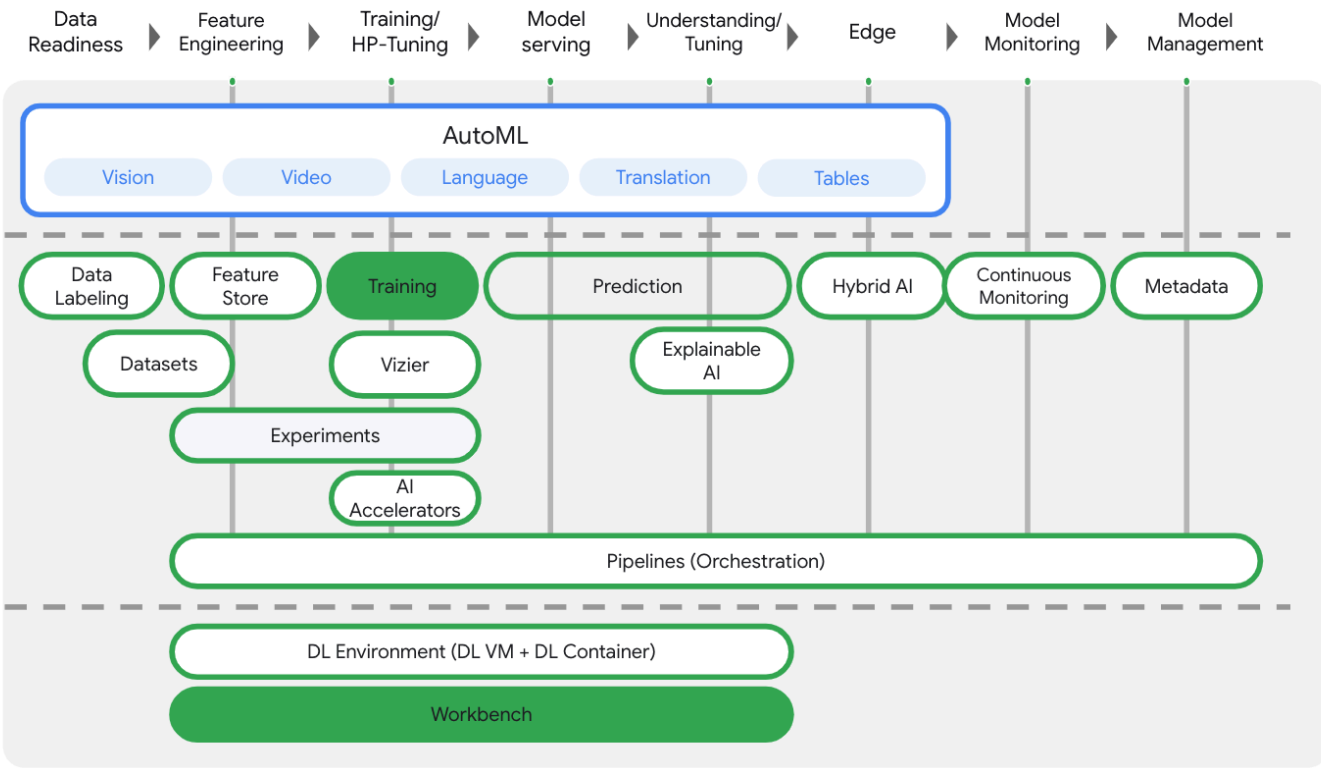
- 使用自訂容器，透過分散式訓練訓練模型
- 啟動多項訓練程式碼試驗，自動調整超參數

在 Google Cloud 上執行這個實驗室的總費用約為 **\$6** 美元。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。[Vertex AI](#) 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。如有任何意見，請參閱[支援頁面](#)。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於「訓練」和「工作台」。



3. 用途總覽

在本實驗室中，您將使用超參數調整功能，找出以 [TensorFlow Datasets](#) 的馬或人資料集訓練的圖片分類模型最佳參數。

超參數調整

使用 [Vertex AI 訓練進行超參數調整](#) 時，系統會根據您選定的超參數，使用符合其指定限制範圍內的值，多次執行訓練應用程式。Vertex AI 會追蹤每個試驗的結果，並調整後續的試驗。

如要搭配 Vertex AI 訓練使用超參數調整功能，您必須對訓練程式碼進行兩項變更：

1. 在主要訓練模組中，為要調整的每個超參數定義指令列引數。
2. 使用這些引數中傳遞的值，在應用程式的程式碼中設定對應的超參數。

分散式訓練

如果您只有一個 GPU，TensorFlow 會使用這個加速器加快模型訓練速度，您不需要額外進行任何作業。不過，如果想透過使用多個 GPU 獲得額外效能提升，則需要使用 `tf.distribute`，這是 TensorFlow 的模組，可跨多個裝置執行運算。

本實驗室使用 `tf.distribute.MirroredStrategy`，您只需變更幾行程式碼，即可將其新增至訓練應用程式。這項策略會在機器上的每個 GPU 建立模型副本。後續的梯度更新會以同步方式進行。也就是說，每個 GPU 會針對不同的輸入資料片段，透過模型計算正向和反向傳遞。然後，系統會透過稱為「全縮減」的程序，彙整所有 GPU 的計算梯度並取平均值。系統會使用這些平均梯度更新模型參數。

您不需要瞭解詳細資料，也能完成本實驗室，但如果想進一步瞭解 TensorFlow 中的分散式訓練運作方式，請觀看下方影片：



步驟 4 · 約 10 分鐘

4. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡的說明](#)操作。

步驟 1：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」（如果尚未啟用）。

步驟 2：啟用 Container Registry API

前往「Container Registry」，然後選取「啟用」（如果尚未啟用）。您將使用這個工具為自訂訓練工作建立容器。

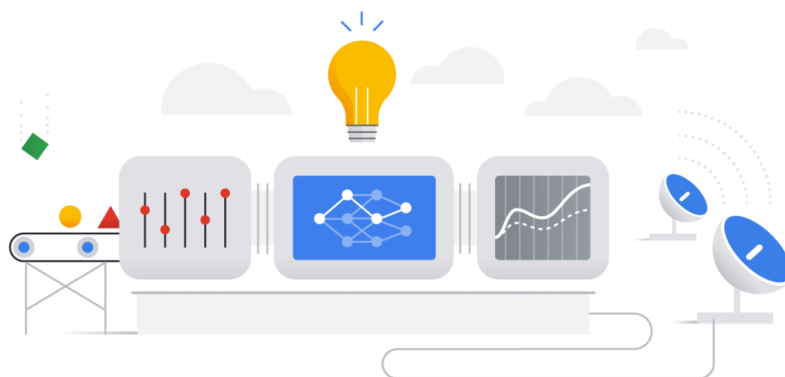
步驟 3：啟用 Vertex AI API

前往 [Cloud 控制台](#) 的 [Vertex AI 專區](#)，然後點選「啟用 Vertex AI API」。

Get started with Vertex AI

Vertex AI empowers machine learning developers, data scientists, and data engineers to take their projects from ideation to deployment, quickly and cost-effectively. [Learn more](#)

ENABLE VERTEX AI API



Region
us-central1 (Iowa)

Prepare your training data

Collect and prepare your data, then import it into a dataset to train a model

+ CREATE DATASET

Train your model

Train a best-in-class machine learning model with your dataset. Use Google's AutoML, or bring your own code.

+ TRAIN NEW MODEL

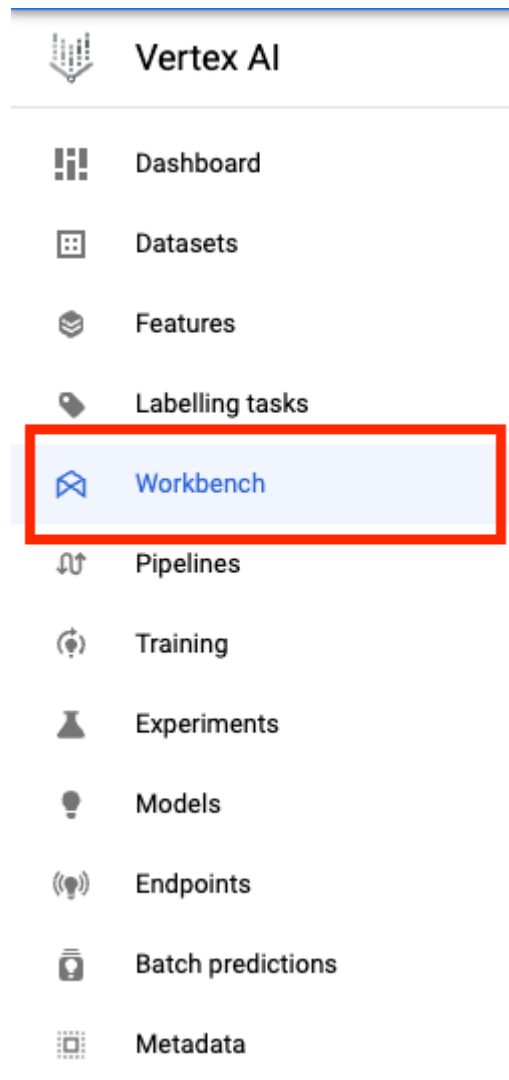
Get predictions

After you train a model, you can use it to get predictions, either online as an endpoint or through batch requests

+ CREATE BATCH PREDICTION

步驟 4：建立 Vertex AI Workbench 執行個體

在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



如果尚未啟用 Notebooks API，請先啟用。



Notebooks API

Google Enterprise API

Notebooks API is used to manage notebook resources in Google Cloud.

ENABLE

TRY THIS API [↗](#)

OVERVIEW

PRICING

DOCUMENTATION

Overview

Notebooks API is used to manage notebook resources in Google Cloud.

Additional details

Type: [SaaS & APIs](#)

Last updated: 15/09/2021

Category: [Google Enterprise APIs](#)

Service name: notebooks.googleapis.com

啟用後，按一下「MANAGED NOTEBOOKS」(代管型筆記本)：

Managed notebooks

[+ NEW NOTEBOOK](#)

[REFRESH](#)

[▶ START](#)

[■ STOP](#)

[⏻ RESET](#)

[🗑 DELETE](#)

[MANAGED NOTEBOOKS](#) [PREVIEW](#)

USER-MANAGED NOTEBOOKS

EXECUTIONS [PREVIEW](#)

SCHEDULES [PREVIEW](#)

Managed notebooks provide JupyterLab services and flexible computing resources integrated with Google Cloud services. [Learn more](#)

然後選取「新增記事本」。

[+ NEW NOTEBOOK](#)

[REFRESH](#)

[▶ START](#)

[MANAGED NOTEBOOKS](#) [PREVIEW](#)

USER-MANAGED NOTEBOOKS

Notebooks service has been moved under the Vertex AI Workbench

為筆記本命名，然後按一下「進階設定」。

my-notebook

Region *

us-central1 (lowa)



費用。

☒ Enable Idle Shutdown

60

Must be integer: 1-600

如果尚未啟用，請在「安全性」下方選取「啟用終端機」。

Security

☒ Enable nbconvert

☒ Enable file downloading from Notebook UI

☒ Enable terminal

其他進階設定可以保留原樣。

接著點選「建立」。執行個體會在幾分鐘內佈建完畢。

建立執行個體後，請選取「Open JupyterLab」。



my-notebook

OPEN JUPYTERLAB

首次使用新執行個體時，系統會要求您驗證身分。按照 UI 中的步驟操作。

Authenticate your managed notebook

To use managed notebooks with your Google Cloud Platform data, you need to grant the Google Cloud SDK permission to access your data and authenticate your managed notebook.

Click the button below to get your authentication code.

Get authentication code 

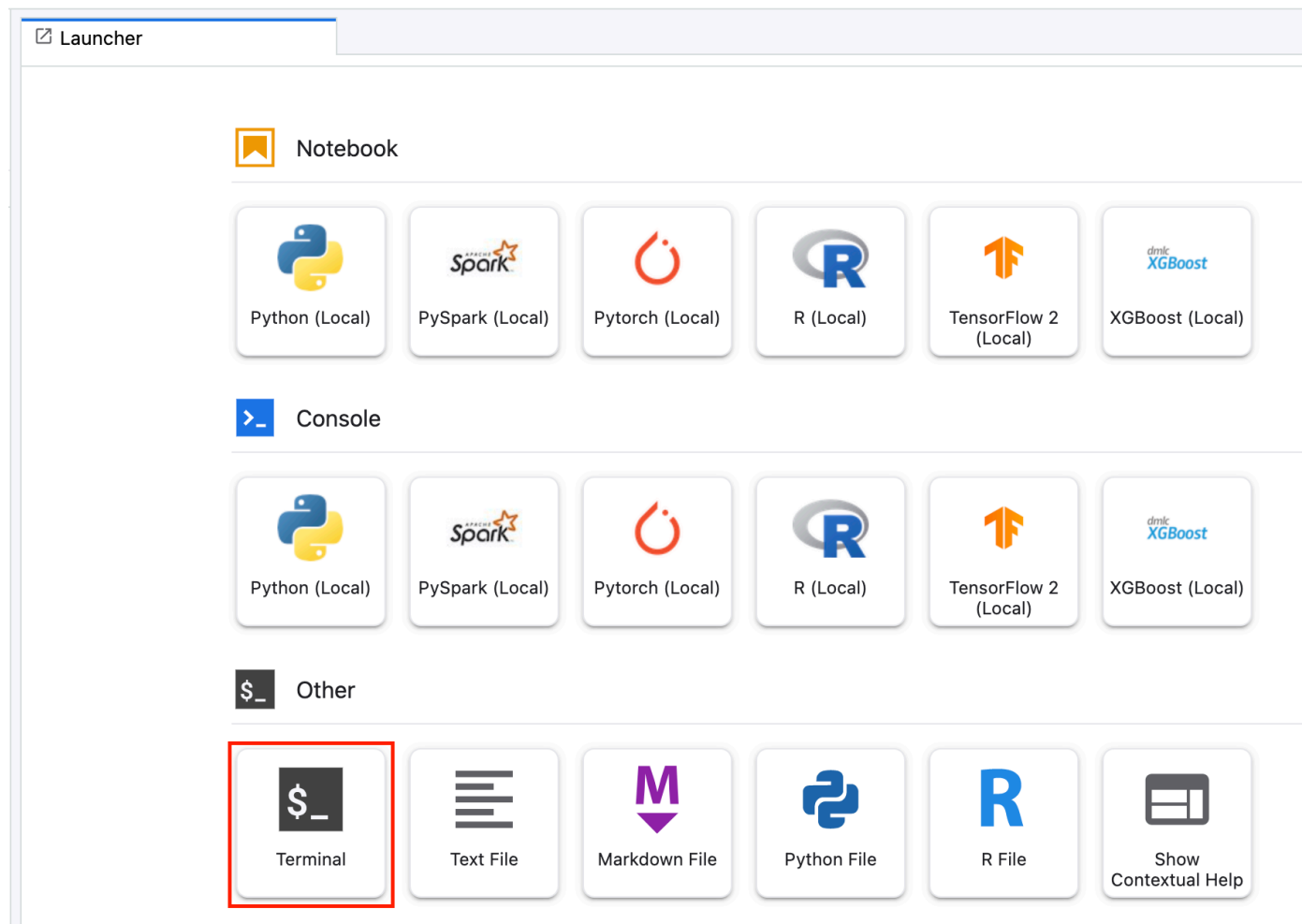
EXIT

AUTHENTICATE

步驟 5 · 約 10 分鐘

5. 編寫訓練程式碼

首先，請從 Launcher 選單開啟筆記本執行個體中的終端機視窗：



建立名為 `vertex-codelab` 的新目錄，然後 `cd` 到該目錄。

```
mkdir vertex-codelab
cd vertex-codelab
```

執行下列指令，為訓練程式碼建立目錄，以及新增程式碼的 Python 檔案：

```
mkdir trainer
touch trainer/task.py
```

`vertex-codelab` 目錄現在應該包含下列項目：

```
+ trainer/
  + task.py
```

接著，開啟剛建立的 `task.py` 檔案，然後貼上以下所有程式碼。

```

import tensorflow as tf
import tensorflow_datasets as tfds
import argparse
import hypertune
import os

NUM_EPOCHS = 10
BATCH_SIZE = 64

def get_args():
    '''Parses args. Must include all hyperparameters you want to tune.'''

    parser = argparse.ArgumentParser()
    parser.add_argument(
        '--learning_rate',
        required=True,
        type=float,
        help='learning rate')
    parser.add_argument(
        '--momentum',
        required=True,
        type=float,
        help='SGD momentum value')
    parser.add_argument(
        '--num_units',
        required=True,
        type=int,
        help='number of units in last hidden layer')
    args = parser.parse_args()
    return args

def preprocess_data(image, label):
    '''Resizes and scales images.'''

    image = tf.image.resize(image, (150,150))
    return tf.cast(image, tf.float32) / 255., label

def create_dataset(batch_size):
    '''Loads Horses Or Humans dataset and preprocesses data.'''

    data, info = tfds.load(name='horses_or_humans', as_supervised=True, with_info=True)

    # Create train dataset
    train_data = data['train'].map(preprocess_data)
    train_data = train_data.shuffle(1000)
    train_data = train_data.batch(batch_size)

    # Create validation dataset
    validation_data = data['test'].map(preprocess_data)
    validation_data = validation_data.batch(batch_size)

```

```

    return train_data, validation_data

def create_model(num_units, learning_rate, momentum):
    '''Defines and compiles model.'''

    inputs = tf.keras.Input(shape=(150, 150, 3))
    x = tf.keras.layers.Conv2D(16, (3, 3), activation='relu')(inputs)
    x = tf.keras.layers.MaxPooling2D((2, 2))(x)
    x = tf.keras.layers.Conv2D(32, (3, 3), activation='relu')(x)
    x = tf.keras.layers.MaxPooling2D((2, 2))(x)
    x = tf.keras.layers.Conv2D(64, (3, 3), activation='relu')(x)
    x = tf.keras.layers.MaxPooling2D((2, 2))(x)
    x = tf.keras.layers.Flatten()(x)
    x = tf.keras.layers.Dense(num_units, activation='relu')(x)
    outputs = tf.keras.layers.Dense(1, activation='sigmoid')(x)
    model = tf.keras.Model(inputs, outputs)
    model.compile(
        loss='binary_crossentropy',
        optimizer=tf.keras.optimizers.SGD(learning_rate=learning_rate, momentum=momentum),
        metrics=['accuracy'])
    return model

def main():
    args = get_args()

    # Create distribution strategy
    strategy = tf.distribute.MirroredStrategy()

    # Get data
    GLOBAL_BATCH_SIZE = BATCH_SIZE * strategy.num_replicas_in_sync
    train_data, validation_data = create_dataset(GLOBAL_BATCH_SIZE)

    # Wrap variable creation within strategy scope
    with strategy.scope():
        model = create_model(args.num_units, args.learning_rate, args.momentum)

    # Train model
    history = model.fit(train_data, epochs=NUM_EPOCHS, validation_data=validation_data)

    # Define metric
    hp_metric = history.history['val_accuracy'][-1]

    hpt = hypertune.HyperTune()
    hpt.report_hyperparameter_tuning_metric(
        hyperparameter_metric_tag='accuracy',
        metric_value=hp_metric,
        global_step=NUM_EPOCHS)

```

```
if __name__ == "__main__":
    main()
```

接著深入瞭解程式碼，並檢查分散式訓練和超參數調整專用的元件。

分散式訓練

1. 在 `main()` 函式中，系統會建立 `MirroredStrategy` 物件。接著，您要在策略範圍內包裝模型變數的建立作業。這個步驟會告知 TensorFlow 哪些變數應在 GPU 間鏡像處理。
2. 批次大小會依 `num_replicas_in_sync` 放大。在 TensorFlow 中使用同步資料平行處理策略時，建議您調整批量大小。如要瞭解詳情，請參閱[這篇文章](#)。

超參數調整

1. 指令碼會匯入 `hypertune` 程式庫。稍後建構容器映像檔時，請務必安裝這個程式庫。
2. 函式 `get_args()` 會為您要調整的每個超參數定義指令列引數。在本範例中，要調整的超參數是學習率、最佳化工具中的動量值，以及模型最後一個隱藏層中的單元數量，但您也可以嘗試其他超參數。然後，系統會使用這些引數中傳遞的值，在程式碼中設定對應的超參數 (例如設定 `learning_rate = args.learning_rate`)。
3. 在 `main()` 函式結尾，系統會使用 `hypertune` 程式庫定義要最佳化的指標。在 TensorFlow 中，Keras `model.fit` 方法會傳回 `History` 物件。`History.history` 屬性會記錄連續訓練週期的訓練損失值和指標值。如果將驗證資料傳遞至 `model.fit`，`History.history` 屬性也會包含驗證損失和指標值。舉例來說，如果您使用驗證資料訓練模型三個訓練週期，並提供 `accuracy` 做為指標，則 `History.history` 屬性會類似於下列字典。

```
{
  "accuracy": [
    0.7795261740684509,
    0.9471358060836792,
    0.9870933294296265
  ],
  "loss": [
    0.6340447664260864,
    0.16712145507335663,
    0.04546636343002319
  ],
  "val_accuracy": [
    0.3795261740684509,
    0.4471358060836792,
    0.4870933294296265
  ],
  "val_loss": [
    2.044623374938965,
    4.100203514099121,
    3.0728273391723633
  ]
}
```

如要讓超參數調整服務找出可將模型驗證準確率提升至最高的超參數值，請將指標定義為 `val_accuracy` 清單的最後一個項目 (或 `NUM_EPOCHS - 1`)。然後將這項指標傳遞至 `HyperTune` 的執行個體。您可以為 `hyperparameter_metric_tag` 選擇任何字串，但稍後啟動超參數調整工作時，需要再次使用該字串。

步驟 6 · 約 15 分鐘

6. 將程式碼容器化

將程式碼容器化的第一步是建立 Dockerfile。Dockerfile 包含執行映像檔所需的所有指令。這會安裝所有必要的程式庫，並設定訓練程式碼的進入點。

步驟 1：編寫 Dockerfile

在終端機中，確認您位於 `vertex-codelab` 目錄，並建立空白 Dockerfile：

```
touch Dockerfile
```

`vertex-codelab` 目錄現在應該包含下列項目：

```
+ Dockerfile
+ trainer/
  + task.py
```

開啟 Dockerfile，然後將下列內容複製到其中：

```
FROM gcr.io/deeplearning-platform-release/tf2-gpu.2-7

WORKDIR /

# Installs hypertune library
RUN pip install cloudml-hypertune

# Copies the trainer code to the docker image.
COPY trainer /trainer

# Sets up the entry point to invoke the trainer.
ENTRYPOINT ["python", "-m", "trainer.task"]
```

這個 Dockerfile 使用 [深度學習容器 TensorFlow 企業版 2.7 GPU Docker 映像檔](#)。Google Cloud 上的深度學習容器預先安裝了許多常見的機器學習和資料科學框架。下載該映像檔後，這個 Dockerfile 會設定訓練程式碼的進入點。

步驟 2：建構容器

在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

```
PROJECT_ID='your-cloud-project'
```

在 Google Container Registry 中，使用容器映像檔的 URI 定義變數：

```
IMAGE_URI="gcr.io/${PROJECT_ID}/horse-human-codelab:latest"
```

設定 Docker

```
gcloud auth configure-docker
```

接著，從 `vertex-codelab` 目錄的根層級執行下列指令，建構容器：

```
docker build ./ -t $IMAGE_URI
```

最後，將其推送至 Google Container Registry：

```
docker push $IMAGE_URI
```

步驟 3：建立 Cloud Storage bucket

在訓練工作中，我們會傳遞暫存 bucket 的路徑。

在終端機中執行下列指令，在專案中建立新的 bucket。

```
BUCKET_NAME="gs://${PROJECT_ID}-hptune-bucket"  
gsutil mb -l us-central1 $BUCKET_NAME
```

步驟 7 · 約 40 分鐘

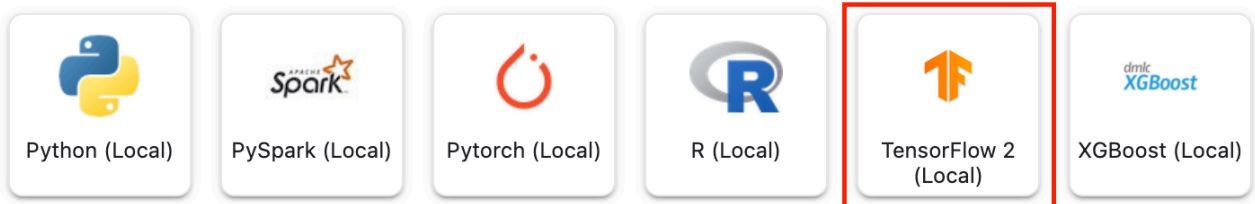
7. 啟動超參數調整工作

步驟 1：建立自訂訓練工作並進行超參數調整

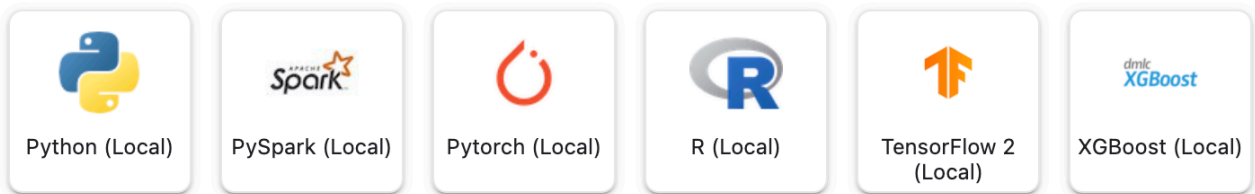
從啟動器開啟新的 TensorFlow 2 筆記本。



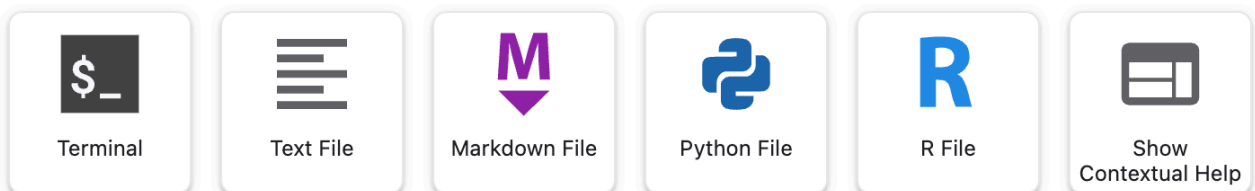
Notebook



Console



Other



匯入 Vertex AI Python SDK。

```
from google.cloud import aiplatform
from google.cloud.aiplatform import hyperparameter_tuning as hpt
```

如要啟動超參數調整工作，您必須先定義 `worker_pool_specs`，指定機型和 Docker 映像檔。以下規格定義了一部搭載兩個 NVIDIA Tesla V100 GPU 的機器。

您需要將 `image_uri` 中的 `{PROJECT_ID}` 換成您的專案。

```
# The spec of the worker pools including machine type and Docker image
# Be sure to replace PROJECT_ID in the "image_uri" with your project.

worker_pool_specs = [{
    "machine_spec": {
        "machine_type": "n1-standard-4",
        "accelerator_type": "NVIDIA_TESLA_V100",
        "accelerator_count": 2
    },
    "replica_count": 1,
    "container_spec": {
        "image_uri": "gcr.io/{PROJECT_ID}/horse-human-codelab:latest"
    }
}]
```

接著定義 `parameter_spec`，這是指定要最佳化參數的字典。字典鍵是您為每個超參數指派的指令列引數字串，字典值則是參數規格。

您需要為每個超參數定義「類型」，以及微調服務會嘗試的值範圍。超參數可以是 `Double`、`Integer`、`Categorical` 或 `Discrete` 類型。如果選取「`Double`」或「`Integer`」類型，則必須提供最小值和最大值。如果選取「類別」或「離散」，則必須提供值。如果是「`Double`」和「`Integer`」類型，您也需要提供「`Scaling`」值。如要進一步瞭解如何選擇最佳比例，請觀看[這部影片](#)。

```
# Dictionary representing parameters to optimize.
# The dictionary key is the parameter_id, which is passed into your training
# job as a command line argument,
# And the dictionary value is the parameter specification of the metric.
parameter_spec = {
    "learning_rate": hpt.DoubleParameterSpec(min=0.001, max=1, scale="log"),
    "momentum": hpt.DoubleParameterSpec(min=0, max=1, scale="linear"),
    "num_units": hpt.DiscreteParameterSpec(values=[64, 128, 512], scale=None)
}
```

最後要定義的規格是 `metric_spec`，這是代表要最佳化指標的字典。字典鍵是您在訓練應用程式程式碼中設定的 `hyperparameter_metric_tag`，值則是最佳化目標。

```
# Dictionary representing metrics to optimize.
# The dictionary key is the metric_id, which is reported by your training job,
# And the dictionary value is the optimization goal of the metric.
metric_spec={'accuracy': 'maximize'}
```

定義規格後，您將建立 `CustomJob`，這是用於在每個超參數調整試驗中執行工作的通用規格。

您需要將 `{YOUR_BUCKET}` 替換為先前建立的 bucket。

```
# Replace YOUR_BUCKET
my_custom_job = aiplatform.CustomJob(display_name='horses-humans',
                                     worker_pool_specs=worker_pool_specs,
                                     staging_bucket='gs://{YOUR_BUCKET}')
```

然後建立並執行 `HyperparameterTuningJob`。

```
hp_job = aiplatform.HyperparameterTuningJob(
    display_name='horses-humans',
    custom_job=my_custom_job,
    metric_spec=metric_spec,
    parameter_spec=parameter_spec,
    max_trial_count=6,
    parallel_trial_count=2,
    search_algorithm=None)

hp_job.run()
```

請注意以下幾個引數：

- `max_trial_count`：您需要為服務執行的測試次數設定上限。一般來說，測試次數越多，結果越好，但會出現報酬遞減的情況，之後再增加測試次數，對您想盡力提升的指標幾乎沒有影響。建議您先進行少量試驗，瞭解所選超參數的影響程度，再擴大規模。
- `parallel_trial_count`：如果使用平行試驗，服務會佈建多個訓練處理叢集。增加平行試驗的數量可縮短超參數微調工作執行時間，但可能會降低整體工作成效。這是因為預設的微調策略會根據先前的試驗結果，在後續試驗中指派值。
- `search_algorithm`：您可以將搜尋演算法設為格線、隨機或預設（無）。預設選項會套用貝氏最佳化方法來搜尋可能的超參數值空間，是建議使用的演算法。如要進一步瞭解這項演算法，請參閱[這篇文章](#)。

工作開始後，您可以在「HYPERPARAMETER TUNING JOBS」（超參數微調工作）分頁中追蹤狀態。

Training + CREATE			
TRAINING PIPELINES		CUSTOM JOBS	<u>HYPERPARAMETER TUNING JOBS</u>
Hyperparameter tuning searches for the best combination of hyperparameter values by optimising metric values across a series of trials. Hyperparameter tuning is only used by custom-trained models and not AutoML models. Learn more Region us-central1 (Iowa) ▼ ?			
Filter Enter a property name			
Name	ID	Status	Job type
horses-humans	1653884290989031424	Running	Hyperparameter tuning job

工作完成後，您可以查看及排序試驗結果，找出最佳超參數值組合。

← horses-humans

[OPEN TENSORBOARD ↗](#)

[VIEW HYPERPARAMETER TUNING JOB INPUTS IN JSON](#)

Hyperparameter tuning trials

Filter

Enter property name or value

☐ Trial ID	accuracy ↓	Training step	learning_rate	momentum	num_units	Log
☐ 1	0.875	10	3.162277660168379e-2	5e-1	128	View logs
☐ 3	0.609	10	6.457449519824135e-3	4.7428523563978187e-1	512	View logs
☐ 2	0.5	10	1.3962109806709977e-1	7.135278165852774e-1	64	View logs
☐ 4	0.5	10	5.8220685582288665e-3	2.550897680672911e-1	64	View logs
☐ 5	0.5	10	1.9037115541004093e-1	7.590541193745313e-1	64	View logs
☐ 6	0.5	10	4.748291131297717e-3	2.2583323522022597e-1	512	View logs

Rows per page: 25 ▾ 1 – 6 of 6 < >

🎉 恭喜！ 🎉

您已瞭解如何使用 Vertex AI 執行下列作業：

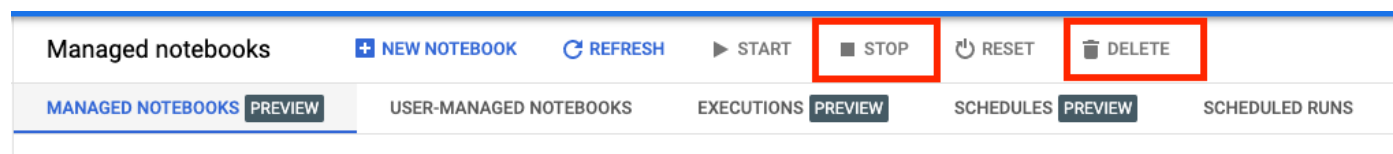
- 執行採用分散式訓練的超參數調整工作

如要進一步瞭解 Vertex AI 的其他部分，請參閱[說明文件](#)。

步驟 8 · 約 1 分鐘

8. 清除所用資源

由於我們將筆記本設定為在閒置 60 分鐘後逾時，因此不必擔心執行個體會關閉。如要手動關閉執行個體，請按一下控制台 Vertex AI Workbench 專區的「停止」按鈕。如要徹底刪除筆記本，請按一下「刪除」按鈕。



如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：

